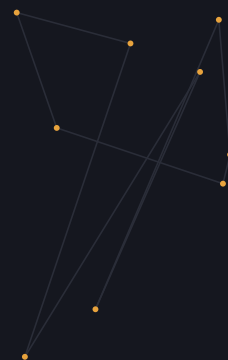


FIELD REPORT · NO. 01

THE LOCAL-FIRST DECADE



Why the next generation of useful software runs on your machine —
not someone else's server.

EXECUTIVE SUMMARY

Computing is quietly re-centering on the device in front of you.

For a decade, “smart software” meant a thin client talking to someone else’s data center. That architecture optimized for the vendor — controllable, meterable, updatable on their schedule. It did not optimize for the user: every feature came with a subscription, a privacy trade, and a dependency on a network connection and a company’s continued goodwill.

A new class of software inverts that bargain. Local-first, agentic tools run primarily on the user’s own hardware — reading their files, controlling their browser, writing their own extensions — and reach out to the network only when they choose to, not by default. The result is software that is faster, more private, and far more durable: it keeps working even if the company that built it disappears tomorrow.



WHAT CHANGED

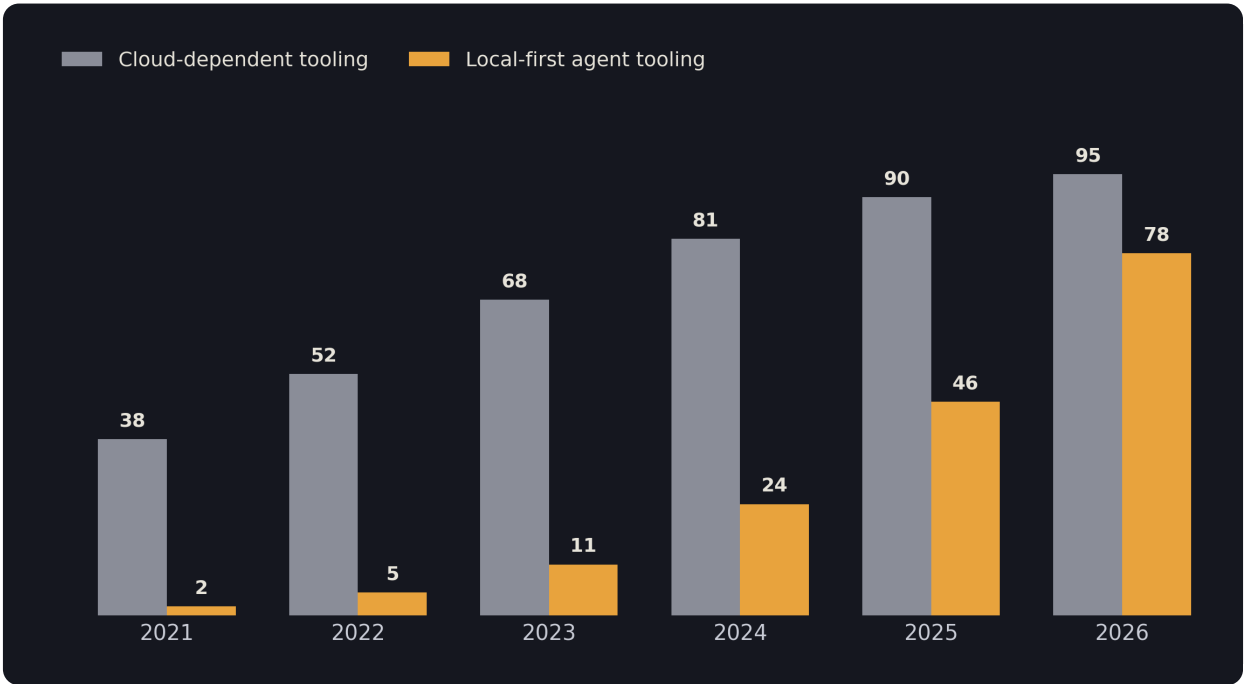
- **Small models got genuinely useful.** A well-tuned 7–9B model running on a consumer GPU now reliably calls tools, follows multi-step plans, and recovers from its own errors — capability that required a frontier API only two years ago.
- **The tool-calling format standardized.** OpenAI-compatible function calling became the lingua franca, so the same agent code now runs unmodified against a local llama.cpp server or a frontier cloud model — swap the endpoint, keep the app.
- **Agents learned to extend themselves.** The newest local frameworks let the model write, test, and load its own tools mid-conversation — turning a fixed feature set into a genuinely open-ended one.

WHO THIS MATTERS FOR

Builders Shipping an agent without a backend bill scaling with every user message.	Privacy-conscious teams Regulated data, source code, or anything that shouldn't transit a third party by default.	Anyone offline often Flights, fieldwork, spotty connections — tools that don't go dark with the network.
--	---	--

ADOPTION CURVE

Local-first tooling is closing the gap fast.



Illustrative adoption index, indexed 0–100 by relative developer mindshare. Local-first tooling started from near-zero in 2021 and has compounded faster than cloud-dependent tooling every year since 2023.

HOW THE TWO MODELS COMPARE

	Cloud-only assistant	Local-first agent
Works offline	✗ No	✓ Yes
Per-seat subscription	✓ Usually required	✗ Not required
Data leaves your machine	✓ By default	✗ Only when you choose
Survives vendor shutdown	✗ No	✓ Yes — it's local
Extends itself with new tools	Rare	✓ Increasingly common
Best-case latency	Network round-trip	Local inference

A note on the comparison above: this isn't an argument that cloud models are obsolete — frontier reasoning still meaningfully outperforms what fits on a consumer GPU. The claim is narrower: for the everyday 90% of tasks — read this file, click that button, remember this preference — local-first is now good enough that routing through someone else's server stops making sense.

PLAYBOOK

Five steps to a real local-first agent.

1

Pick a tool-calling-capable local model

Anything ~7B+ with OpenAI-style function calling. Verify it actually emits structured tool calls — some chat-tuned models narrate intent instead of calling the function, which silently breaks every downstream step.

2

Give it a small, sharp toolset first

File read/write, shell, web fetch. Resist the urge to ship 80 tools on day one — each tool definition costs context budget on every single turn, whether it's used or not.

3

Sandbox writes, leave reads open

Let the agent read broadly so it can actually understand the user's machine; confine writes/deletes/shell commands to an explicit workspace with a permission gate the user can set to always-allow once trust is earned.

4

Let it extend itself

A ``create_plugin``-style tool — write code, validate it, load it, use it the same turn — turns a fixed feature set into one that grows with the user's actual needs instead of waiting on the next release.

5

Make the cloud an opt-in ceiling, not the floor

Keep the default path 100% local. Let a pasted API key swap the same agent onto a frontier model for the hard 1% of tasks — same tools, same memory, no rewrite.

The pattern that keeps showing up: the agents that feel most alive aren't the ones with the most built-in tools — they're the ones that can notice a gap and close it themselves, mid-conversation, without waiting for a human to ship an update.

COMMON PITFALLS

- **Shipping every tool to every turn.** Tool schemas cost real context budget whether the model uses them or not — a sprawling toolset quietly eats the room needed for the actual conversation.
- **Confining reads as tightly as writes.** An agent that can't see the user's real files and folders ends up guessing — and guessing is where hallucinated paths and wasted turns come from.
- **Treating the cloud fallback as an afterthought.** The smoothest local-first tools make switching to a frontier model a one-line config change, not a second codebase.

“The real shift isn't local models getting smarter than cloud ones. It's that they finally got good enough — and good enough, running on hardware you already own, beats brilliant behind someone else's paywall.”

— Field Report editorial note

TAKEAWAY

Build for the machine in front of you.

The cloud will still be there for the hard 1%. Everything else belongs on-device.

2027

PROJECTED CROSSOVER
POINT

9 months

TYPICAL TIME-TO-FIRST
USEFUL LOCAL AGENT

Day one

WHEN CLOUD FALLBACK
SHOULD BE WIRED IN